

When a C program is compiled and executed, its memory is divided into different segments.

Understanding these segments and their overlaps is crucial for efficient programming and avoiding memory-related issues. In this post, we will discuss the four main segments of a C program's memory and their overlaps.

1. Text segment:

The text segment, also known as the code segment, is where the compiled code of the program resides. This segment is read-only and contains the program's instructions, including all the functions, statements, and declarations. The text segment is usually located at the lowest memory address and is not shared with other programs.

2. Data segment:

The data segment is where initialized global and static variables are stored. This segment is writable, and it contains data that is accessed and modified during the program's execution. The data segment is usually located after the text segment and before the heap and stack segments.

3. Heap segment:

The heap segment is used for dynamic memory allocation. When a program needs to allocate memory during runtime, it requests it from the heap segment. This segment is also writable and grows dynamically as more memory is allocated. The heap segment is located above the data segment and can overlap with the stack segment.

4. Stack segment:

The stack segment is used for storing function call frames and local variables. This segment grows and shrinks dynamically as functions are called and return. The stack segment is usually located at the top of the memory address space and grows downwards. The stack segment can overlap with the heap segment if they grow towards each other.

In summary, a C program's memory is divided into text, data, heap, and stack segments. These segments overlap in some cases, and understanding their interaction is crucial for efficient programming and avoiding memory-related issues. By managing these segments effectively, programmers can ensure their programs run smoothly and efficiently.

